

Rapport des APIs manquantes

Fonctionnalité : Administration — SUPER_ADMIN vs ADMIN normal

Date du rapport : 10/06/2026

1. Contexte

La plateforme distingue deux types d'administrateurs via le champ `niveau_acces` (enum `NiveauAcces`) dans la table `Administrateur` : `SUPER_ADMIN` et `ADMIN`. Les `SUPER_ADMIN` ont `ecole = null`, les `ADMIN` ont une école définie. Le frontend a été modifié pour détecter le type via `ecole === null` et adapter l'affichage (global vs filtré par établissement).

Cependant, certaines fonctionnalités ne peuvent pas être implémentées correctement sans des endpoints API supplémentaires. Ce rapport liste ces APIs manquantes.

1. Historique des actions filtré par établissement

Page(s) concernée(s) : `/admin/historique`

Fonctionnalité bloquée : Un administrateur normal (`ADMIN`) ne peut voir que l'historique de son propre établissement. Actuellement, `GET /api/historique-actions/` retourne toutes les actions, sans filtre.

Endpoint requis : `GET /api/historique-actions/`

Méthode HTTP : `GET`

Paramètres / Body requis : `?ecole=NOM_ECOLE` (query param optionnel). Si absent, retourne tout (pour `SUPER_ADMIN`). Si présent, filtre par l'école de l'utilisateur lié à chaque action.

Structure de réponse attendue : Tableau d'objets `HistoriqueAction`, incluant les données de l'utilisateur (avec son école). Les champs : `id_historique`, `id_utilisateur`, `type_action`, `entite_concernee`, `details`, `date_action`, `utilisateur`:
{ `id_utilisateur`, `nom`, `prenom`, `email`, `ecole` }.

Pourquoi cette API est nécessaire : Le endpoint `GET /api/historique-actions/` ne supporte actuellement aucun filtre par école. Le service (`recupererToutesLesActions`) passe les query params directement à Prisma.where, mais le modèle `HistoriqueAction` n'a pas de champ école. Il faudrait un endpoint qui joint sur `utilisateur.ecole`.

Exemple de requête : `GET /api/historique-actions/?ecole=ENSATanger`

Exemple de réponse : {

```
"success": true,
"data": [
  {
    "id_historique": "abc123",
    "id_utilisateur": "user1",
    "type_action": "VALIDATION",
    "entite_concernee": "Activite",
    "details": "Activité validée",
    "date_action": "2026-06-10T10:00:00Z",
    "utilisateur": {
      "id_utilisateur": "user1",
      "nom": "Dupont",
      "prenom": "Jean",
      "email": "j.dupont@example.com",
      "ecole": "ENSATanger"
    }
  }
]
```

2. Niveau d'accès (niveau_acces) dans la réponse de /auth/me et /auth/login

Page(s) concernée(s) : Toutes les pages admin

Fonctionnalité bloquée : Détection fiable du type d'administrateur. Actuellement, le frontend utilise `ecole === null` pour détecter `SUPER_ADMIN` (car la validation backend impose `ecole=null` pour `SUPER_ADMIN`). Cette méthode

est indirecte et pourrait être cassée si la validation change.

Endpoint requis : /api/auth/me et /api/auth/login

Méthode HTTP : GET / POST

Paramètres / Body requis : Aucun paramètre supplémentaire. Il suffit d'inclure le champ niveau_acces dans l'objet utilisateur retourné.

Structure de réponse attendue : L'objet utilisateur doit inclure : { id_utilisateur, nom, prenom, email, role, ecole, niveau_acces: "SUPER_ADMIN" | "ADMIN", status_compte, ... }

Pourquoi cette API est nécessaire : Le endpoint getMe() dans authentication.service.js ne fait pas d'inclure sur la relation administrateur. Il retourne seulement les champs de Utilisateur. Le champ niveau_acces est stocké dans Administrateur.niveau_acces et n'est pas inclus dans la réponse. Le frontend doit actuellement inférer le type via ecole.

Exemple de requête : GET /api/auth/me

Exemple de réponse : {

```
"success": true,
"data": {
  "id_utilisateur": "admin1",
  "nom": "Super",
  "prenom": "Admin",
  "email": "super@uae.ac.ma",
  "role": "ADMINISTRATEUR",
  "ecole": null,
  "niveau_acces": "SUPER_ADMIN",
  "status_compte": "ACTIF"
}
```

3. Liste des établissements (écoles)

Page(s) concernée(s) : /admin/dashboard (configuration super admin)

Fonctionnalité bloquée : Super admin : voir et gérer la liste de tous les établissements. Actuellement, il n'existe aucun endpoint qui retourne la liste des écoles (enum Ecole).

Endpoint requis : GET /api/etablissements/

Méthode HTTP : GET

Paramètres / Body requis : Aucun paramètre requis. Accessible uniquement aux SUPER_ADMIN (ou ADMINISTRATEUR).

Structure de réponse attendue : Tableau d'objets : { id_ecole?: string, nom: string, code: string, ... }

Pourquoi cette API est nécessaire : L'enum Ecole est défini dans Prisma (ENSATanger, ENCGTanger, FSTTanger, etc.) mais aucune API ne l'expose. Pour qu'un SUPER_ADMIN puisse gérer les établissements ou filtrer par établissement, une API dédiée est nécessaire.

Exemple de requête : GET /api/etablissements/

Exemple de réponse : {

```
"success": true,
"data": [
  { "nom": "ENSATanger" },
  { "nom": "ENCGTanger" },
  { "nom": "FSTTanger" }
]
```

4. File de validation des activités pour SUPER_ADMIN (tous établissements)

Page(s) concernée(s) : /admin/verifications — SUPER_ADMIN

Fonctionnalité bloquée : Le SUPER_ADMIN ne peut pas voir toutes les activités en attente de validation via le endpoint dédié /activites/a-valider car le backend (recupererActivitesAValider) retourne [] si ecole == null.

Endpoint requis : GET /api/activites/a-valider?scope=all ou nouveau endpoint GET /api/activites/en-attente

Méthode HTTP : GET

Paramètres / Body requis : Paramètre optionnel : ?scope=all (pour SUPER_ADMIN). Ou un paramètre ?status_validation=EN_ATTENTE sur le endpoint /activites/.

Structure de réponse attendue : Tableau d'activités avec statut EN_ATTENTE, incluant étudiant, son école, et les détails de l'activité.

Pourquoi cette API est nécessaire : Le service recupererActivitesAValider() (activiteParascolaire.service.js:170) récupère l'école de l'admin connecté et retourne [] si ecole est null. Actuellement le frontend SUPER_ADMIN utilise le endpoint GET /api/activites/?status_validation=EN_ATTENTE comme workaround, mais ce endpoint n'est pas conçu pour cela et peut causer des incohérences.

Exemple de requête : GET /api/activites/?status_validation=EN_ATTENTE

Exemple de réponse : {

```
"success": true,
"data": [
  {
    "id_activite": "act1",
    "nom_activite": "Hackathon 2026",
    "status_validation": "EN_ATTENTE",
    "etudiant": {
      "id_etudiant": "et1",
      "utilisateur": {
        "nom": "El Idrissi",
        "prenom": "Ahmed",
        "email": "a.elidrissi@example.com",
        "ecole": "ENSATanger"
      }
    }
  }
]
```

Résumé

Les modifications frontend suivantes ont été implémentées SANS toucher au backend :

1. Détection SUPER_ADMIN via ecole === null (getters isSuperAdmin / isNormalAdmin dans authStore)
2. adminStore.js : utilisation de GET /api/etudiants/ et GET /api/professeurs/ pour SUPER_ADMIN (endpoints globaux existants)
3. adminStore.js : utilisation de GET /api/professionnels/en-attente et PATCH /api/professionnels/:id/valider (endpoints existants)
4. adminStore.js : pour la file d'attente des activités SUPER_ADMIN, workaround via GET /api/activites/?status_validation=EN_ATTENTE
5. AdminVerifications.vue : section professionnels visible uniquement pour SUPER_ADMIN
6. AdminDashboard.vue : indicateur de scope (global vs établissement) + file professionnels
7. AdminUsers.vue / AdminPortfolios.vue : colonne Établissement visible uniquement pour SUPER_ADMIN
8. AdminSideBar.vue : badge "Super Admin" / "Admin"

Les APIs documentées ci-dessus sont nécessaires pour une implémentation complète et robuste.